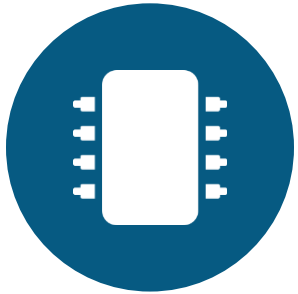

Total Bandwidth Server

Moshiour Rahman Prince

Dept. of Electronic Engineering, Hamm-Lippstadt University of Applied Sciences.

Outline

- Background: EDF and the utilization bound
- The Total Bandwidth Server: core idea and deadline rule
- Comparison with other aperiodic servers
- UPPAAL model and application example
- Experimental results and formal verification
- Discussion and conclusion

Background: EDF and the Utilization Bound

HOCHSCHULE HAMM-LIPPSTADT

- EDF (Earliest Deadline First): dynamic priority, always run nearest deadline
- Optimal on a uniprocessor: if any algorithm can schedule the set, EDF can
- Schedulable under EDF if $U_p = \Sigma C_i / T_i \leq 1$
- RM (Rate Monotonic): fixed priority $\propto$ 1/period; bound only $\ln 2 \approx 69.3\%$
- EDF reaches full 100% utilization — more efficient than RM

$U_p = \Sigma (C_i / T_i) \leq 1$ → EDF schedulability bound

The Total Bandwidth Server — Core Idea

- Proposed by Spuri & Buttazzo (1996), within the EDF framework
- Reserve a fraction U_s of the processor for aperiodic tasks
- A virtual server — no time slots, no replenishment queue
- On each aperiodic arrival, assign a synthetic absolute deadline
  - The deadline places the job in the EDF queue alongside periodic tasks
  - Long-run aperiodic CPU consumption is bounded by U_s
- Bandwidth-optimal and extremely lightweight at runtime

TBS Deadline-Assignment Rule

HOCHSCHULE HAMM-LIPPSTADT

$$d_k = \max(r_k , d_{k-1}) + C_k / U_s \qquad (d_0 = 0)$$

- d_k : synthetic deadline assigned to the k-th aperiodic job
- r_k : arrival (release) time of the k-th job
- d_{k-1} : deadline assigned to the previous aperiodic job
- C_k : worst-case execution time of the k-th job
- U_s : server utilization (bandwidth reservation), $0 < U_s < 1$
- max keeps deadlines non-decreasing → aperiodic use ≤ $U_s \cdot \Delta t$ in any interval

Comparison with Other Aperiodic Servers

HOCHSCHULE HAMM-LIPPSTADT

Server	Characteristic vs. TBS
Polling Server	Wastes bandwidth if no aperiodic pending
Deferrable Server	Preserves capacity, but extra schedulability cost
Sporadic Server	Needs replenishment bookkeeping; more complex
CBS	Handles overruns / unknown WCET; heavier mechanism
TBS	O(1), no waste — spare capacity flows to periodics

UPPAAL Model Architecture

HOCHSCHULE HAMM-LIPPSTADT

PeriodicGenerator

Releases periodic jobs each period

AperiodicGenerator

Assigns TBS synthetic deadline

Scheduler (CPU)

EDF select, 1-unit step, miss check

- Network of timed automata; one generator process per task (parameterized)
- Discrete time: integer currentTime; cpuClock measures one unit per cycle
- Committed chain MissCheck → Release → Select → Dispatch → Run / Idle
- Integer arithmetic reproduces the exact preemptive EDF schedule

Application Example

- Periodic tasks: $\tau_0 = (1, 4)$, $U_0 = 1/4$ and $\tau_1 = (2, 6)$, $U_1 = 1/3$
- Periodic load: $U_p = 1/4 + 1/3 = 7/12 \approx 0.58$
- Server bandwidth: $U_s = 1/4 \rightarrow C_k / U_s = 4 \cdot C_k$
- Combined: $U_p + U_s = 7/12 + 3/12 = 5/6 \approx 0.83 \leq 1$ ✓
- Three aperiodic jobs: J_0 at t=2 (C=1), J_1 at t=8 (C=1), J_2 at t=14 (C=2)
- Observation window: HORIZON = 24 time units

Worked Synthetic Deadlines

HOCHSCHULE HAMM-LIPPSTADT

Job	r_k	C_k	C_k/U_s	d_k
J_0	2	1	4	6
J_1	8	1	4	12
J_2	14	2	8	22

- $d_k = \max(r_k, d_{k-1}) + C_k/U_s$ applied with $U_s = 1/4$
- Deadlines serialized non-decreasing: 6 → 12 → 22, placed in the EDF queue
- Integrated EDF schedule over [0, 24): every deadline met, CPU idle 6 of 24 units

Experimental Results

Job	Arrival	Finish	Resp.	d_k	Met?
J_0	2	3	1	6	yes
J_1	8	9	1	12	yes
J_2	14	18	4	22	yes

- All response times far below synthetic deadlines; deadlineMiss stays false
- Utilization over window: periodic 14/24, aperiodic 4/24 (< U_s=0.25), idle 6/24
- Unused server capacity flows to the periodic set rather than being wasted
- All 7 TCTL verification queries hold (deadlock-free, no miss, deadlines = 6/12/22)

Continuous model — verified over unbounded time, all sporadic arrival patterns:

- A[] not deadlock — scheduler runs forever
- A[] not deadlineMiss — NO deadline ever missed, over all time, for every arrival pattern (now a genuine proof, not one-window evidence)
- A[] not aperiodicOverflow — bounded job pool never overflows
- A[] serverSlack ≤ C_k/U_s — TBS throttles aperiodic work to at most U_s
- aperiodicActive[0] --> not aperiodicActive[0] — liveness: every request eventually served, never starved

Discussion

HOCHSCHULE
HAMM-LIPPSTADT

Advantages

- O(1) overhead — fits constrained embedded systems
- Inherits EDF optimality
- No bandwidth wasted vs. Polling / Deferrable servers
- Spare capacity flows to periodic tasks

Limitations

- WCET C_k must be known at arrival time
- Handles only soft aperiodic tasks (synthetic deadlines)
- Distributed end-to-end deadlines need extra mechanisms
- Continuous UPPAAL model now proves no deadline miss over unbounded time for all sporadic arrivals; bounded run remains illustrative

Conclusion

HOCHSCHULE
HAMM-LIPPSTADT

- TBS elegantly integrates aperiodic tasks into EDF-scheduled systems
- Deadline rule $d_k = \max(r_k, d_{k-1}) + C_k/U_s$ bounds bandwidth with O(1) overhead
- Simple schedulability condition: $U_p + U_s \leq 1$
- UPPAAL model (3 templates, discrete-time EDF) verified deadlock-free and miss-free
- Assigned deadlines match theory (6, 12, 22); aperiodic share within reserved U_s
- Future work: distributed precedence constraints; compare against CBS

References

[1] M. Spuri and G. C. Buttazzo, “Scheduling Aperiodic Tasks in Dynamic Priority Systems,” Real-Time Systems, vol. 10, no. 2, pp. 179–210, 1996.

[2] C. L. Liu and J. W. Layland, “Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment,” J. ACM, vol. 20, no. 1, pp. 46–61, 1973.

[3] G. C. Buttazzo, Hard Real-Time Computing Systems, 3rd ed. Springer, 2011.

[4] H. Kopetz, Real-Time Systems: Design Principles for Distributed Embedded Applications, 2nd ed. Springer, 2011.

[5] A. David, K. G. Larsen, A. Legay, M. Mikučionis, D. B. Poulsen, “UPPAAL SMC Tutorial,” Int. J. STTT, vol. 17, no. 4, pp. 397–415, 2015.

[6] L. Abeni and G. Buttazzo, “Integrating Multimedia Applications in Hard Real-Time Systems (CBS),” Proc. IEEE RTSS, 1998.